Summary and Sentiment Prediction of News Article

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

By

C Darshita

19BCE2246

Korrapati Rishitha

19BCE2275

Mopuru Lahari

19BBS0037

Under the guidance of
Prof. / Dr.M Rajasekhara Babu

SCOPE

VIT, Vellore.

May, 2023

DECLARATION

I hereby declare that the thesis entitled "Summary and Sentiment Prediction of News Article" submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. Dr. M Rajasekhara Babu.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 19/5/2023

Signature of the Candidate

C Darshita (19BCE2246)

Korrapati Rishitha (19BCE2275)

Mopuru Lahari(19BBS0037)

CERTIFICATE

This is to certify that the thesis entitled "Summarization and Sentiment Prediction of News Article" submitted by **C Darshita & 19BCE2246**, **Korrapati Rishitha & 19BCE2275**, **Mopuru Lahari & 19BBS0037**, **SCOPE**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by him / her under my supervision during the period, 01. 07. 2022 to 30.04.2023, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date : 19/05/2023 **Signature of the Guide**

Internal Examiner **External Examiner**

Dr. VAIRAMUTHU S

Computer Science and Engineering

Dr. Priya G

Computer Science and Engineering with specialization in Business Systems

ACKNOWLEDGEMENTS

We take this platform in expressing our gratefulness to everyone at VIT for being our pillar of strength throughout our **B.Tech**. To begin with, we would like to convey our gratitude to our mentor, Professor **Dr. M Rajasekhara Babu**, for being our guide. His timely recommendations and guidance have steered our thesis to get escalated to the next higher level. His incessant and practical advises, perspective and thought promoting comments were the guiding lights for us. Presence of the element of his immense knowledge and patience is vivid all through our thesis -Summary and Sentiment analysis of News Article. His professional expertise and boundless knowledge scaffolded our thesis. Imagining our thesis without his relentless help isn't possible to us.

We sincerely would like to express our gratitude **SCOPE** School, **Vellore Institute of Technology (VIT)** for extending continuous support throughout the four years of B. Tech in developing the abilities in us.

Our gratefulness and humble thanks to our parents whose unfailing support has encouraged at all the stages of our studentship.

Student Name

C Darshita (19BCE2246)

Korrapati Rishitha (19BCE2275)

Mopuru Lahari(19BBS0037)

Executive Summary

In today's fast-paced world, many people have shorter attention spans and may not have the time or patience to read through long articles. They may be looking for quick and concise news updates that they can digest quickly and move on. We need to provide a much more effective way of presenting news on digital news platforms.

The news covers stories with mixed emotions and we can assess public sentiment and emotions with the use of sentiment analysis. With the help of sentiment analysis algorithms and visualization techniques, we can analyze the sentiment of the articles. This can help the user grasp the sentiment of the news article quickly.

Short summaries of news articles could be provided to allow users to skim through them instead of reading long articles. Text summarization techniques can be used to condense longer texts that are often difficult to read.

Our paper provides a comparative analysis of different types of text summarization and sentiment analysis algorithms that can be used to create a news web application that automatically summarizes the selected news article and indicates its sentiment. It also provides visualization technique for easy understating of the news article.

CONTENTS

		Page No.
	Acknowledgement	4
	Executive Summary	5
	Table of Contents	6
	List of Figures	8
	List of Tables	9
1	**INTRODUCTION**	**10**
	1.1 Theoretical Background	10
	1.2 Motivation	11
	1.3 Aim of the Proposed Work	11
	1.4 Objective(s) of the Proposed Work	11
2.	**Literature Survey**	**12**
	2.1. Survey of the Existing Models/Work	12
	2.2. Summary/Gaps identified in the Survey	14
3.	**Overview of the Proposed System**	**17**
	3.1. Introduction and Related Concepts	17
	3.2. Framework, Architecture or Module for the Proposed System	17
	3.3. Proposed System Model (ER Diagram/UML Diagram/Mathematical Modeling)	28
4.	**Proposed System Analysis and Design**	**32**
	4.1. Introduction	32
	4.2. Requirement Analysis	32
	4.2.1.Functional Requirements	
	4.2.1.1. Product Perspective	
	4.2.1.2. Product features	
	4.2.1.3. User characteristics	
	4.2.1.4. Assumption & Dependencies	
	4.2.1.5. Domain Requirements	
	4.2.1.6. User Requirements	
	4.2.2.Non-Functional Requirements	34
	4.2.2.1. Product Requirements	
	4.2.2.1.1. Efficiency (in terms of Time and Space)	
	4.2.2.1.2. Reliability	
	4.2.2.1.3. Portability	
	4.2.2.1.4. Usability	

4.2.2.2. Organizational Requirements 30
4.2.2.2.1. Implementation Requirements (in terms of deployment)
4.2.2.2.2. Engineering Standard Requirements
4.2.3.System Requirements 35
4.2.3.1. H/W Requirements (details about Application Specific Hardware)
4.2.3.2. S/W Requirements (details about Application Specific Software)
5. Results and Discussion 36
6. References 46
APPENDIX A

List of Figures

Figure No	Title	Page no
3.1	Architecture of sentiment analysis	17
3.2	Lexicon based approach	18
3.3	TextBlob	19
3.4	VADER	19
3.5	Text summarization with TF-IDF	20
3.6	Text summarization with LSA	21
3.8	Pegasus for text summarization	22
3.9	ProphetNet for text summarization	24
3.10	Various statistical methods	25
3.11	Common transformers model	25
3.12	System architecture for web app	26
3.3.1	Use case diagram	28
5.1	Sentence-polarity	36
5.2	Pos, Neg,Neu scores	37
5.3	Article’s score(TextBlob)	38
5.4	Pos, Neg,Neu scores	38
5.5	Article’s score(VADER)	38
5.6	Article’s score (BERT)	39
5.7	TextBlob comparison	42
5.8	VADER comparison	42
5.9	Web app structure	43
5.10	Choose news category	43
5.11	News poster	44
5.12	Sentiment score, summary	44
5.13	Word Cloud	45
5.14	Link, date and time	45

List of Tables

Table No	Title	Page no
2.1	Lit Summary	14
3.3.1	Description of user	28
3.3.2	Description of system	29
3.3.3	Use case table of News category	29
3.3.4	Use case table of News article fetch	29
3.3.5	Use case table of No of articles	30
3.3.6	Use case table of News title, poster	30
3.3.7	Use case table of News sentiment score	30
3.3.8	Use case table of News summary	31
3.3.9	Use case table of News article link	31
3.3.10	Use case table of News article published data and time	31
4.2.1	Functional requirements	32
4.2.1.1	User characteristics	33
5.1	Sentiment scores obtained using various techniques	39
5.2	Summary obtained using various techniques	40
5.3	Rouge values	41

1. INTRODUCTION

1.1. Theoretical Background

The development of modern technology has altered traditional lifestyle in several ways. The speed at which news and events are published online has increased, thanks to the development of information technology (IT). Massive volumes of data are constantly being published on the internet by millions of people in the form of comments, blogs, news sharing websites, social media microblogging apps, and many other forms. Since manually navigating through such vast amounts of data is difficult, sophisticated technologies have been developed to carry out this operation quickly and automatically

The news covers stories with mixed emotions, including positive, negative, and neutral. We can assess public sentiment and emotions with the use of sentiment analysis, and we can also gather important context-specific data. Sentiment analysis is the process of evaluating data and classifying it in line with the needs of the investigation.

News articles written by journalists are usually presented in the form of long and extensive pieces. In our fast-paced world, individuals often have limited attention spans, online reading habits differ from those of print readers, with users often resorting to skimming and scanning content to locate the desired information swiftly. So, this format often poses a challenge, the excessive length, which makes it difficult for readers to maintain focus and interest, making it less engaging.

With the help of modern text summarization algorithms, we generate short comprehensive summaries of these long news articles quickly. Summaries can provide a brief overview of the main points and key takeaways from the article, allowing readers to decide whether they want to read the full story.

1.2. Motivation

The sentiment prediction of a news article can give us a quick assessment of the overall sentiment or tone of the news article. It can act as a visual cue or summary that helps users quickly identify articles of interest. Users can scan through the list of articles and focus on those with sentiment scores aligning with their preferences.

News summarization makes it easier for the users to go through daily news articles quicker, saving their precious time. By including a summary, digital news platforms can improve the user experience for their readers which can be especially important as many people consume news on their mobile devices and may not have the time or attention span to read lengthy articles .

By displaying sentiment scores and short summaries for each news article , we can make the news browsing experience more interactive and engaging. This can help news platforms attract and retain readers. Improve the user experience and boost the visibility in search engines.

1.3. Aim of the Proposed Work

Our aim is to develop a web app which can analyse the sentiment of a given article, provide its summary and visualize the article with the help of data visualization techniques, so that user can have a better and easy/instant analysis of the information. Meanwhile in this process, we aim to compare different models and algorithms to choose the best methods for each task which suits our final product.

1.4. Objective(s) of the Proposed Work

• Providing a system that summarizes the news article which helps people to quickly understand the content of article.

• Calculating the sentiment score to give an overall feature of the article i.e., positive, negative, or neutral emotion.

• Data visualization techniques such as word clouds will be efficient in summarizing the key words.

• To analyze the performance over number various news articles.

2. LITERATURE SURVEY

2.1. Survey of the Existing Work:

Researchers have contributed several works in the field of sentiment analysis [1],[2],[3],[4],[5],[6] and text summarization [7],[8],[9],[10],[11],[12]. In recent years, several models have been developed to perform sentiment analysis with high accuracy and efficiency. This survey focuses on the popular models of sentiment analysis and provides an overview of their methodologies, strengths, and applications. It also provides an overview of various text summarization techniques, including both extractive (TextRank, TF-IDF, LSA) and abstractive (Pegasus, ProphetNet, BART) approaches, highlighting their key features and references for further exploration.

The research by Neogi et al. (2021) [1] showcases the application of sentiment analysis, specifically in the context of analyzing sentiment during Indian farmers' protests using Twitter data. Their work demonstrates the effectiveness of sentiment analysis techniques in capturing sentiment dynamics and classifying sentiment polarity. BERT is particularly effective for sentiment analysis due to its ability to capture contextual information and semantic relationships in text [6].

Another model, VADER is a rule-based sentiment analysis model specifically designed for social media text. It incorporates a combination of lexical, grammatical, and syntactical heuristics to identify sentiment polarity and intensity [6]. VADER has proven to be effective in capturing sentiment nuances, especially in short and informal text.

The study by Souma et al. (2019) [2] demonstrates the application of sentiment analysis using deep learning methods and explores the predictive power of historical news sentiments based on financial market performance to forecast financial news sentiments., showcasing its ability to handle diverse textual sources and provide accurate sentiment classification.

The work by Guerra et al. (2021) [3] demonstrates the application of TextBlob in sentiment analysis of customer reviews, highlighting its effectiveness in providing sentiment polarity scores and subjectivity analysis for opinion mining.

To provide a comprehensive analysis, a comparative study by Li et al. (2021) [6] evaluated the performance of BERT, VADER, and TextBlob in sentiment analysis tasks. The study compared the accuracy, precision, recall, and F1 score of each model using various datasets. The findings indicated that BERT achieved the highest performance in terms of accuracy and F1 score, followed by VADER, while TextBlob showed relatively lower performance.

However, TextBlob excelled in terms of simplicity and ease of use. The research by Rada et al. presents an application of the TextRank algorithm to the task of text summarization [7]. They introduce TextRank, a graph-based ranking algorithm inspired by PageRank, which identifies important sentences in a document for extractive summarization. The study showcases the scalability and simplicity of TextRank and its potential for diverse domains and languages.

Zhai and Lafferty (2001) [8] investigate the application of TF-IDF weighting for automatic term extraction. They introduce the concept of TF-IDF, which considers the frequency of terms in a document as well as their rarity across the corpus. The study highlights the effectiveness of TF-IDF weighting in capturing term importance and its potential applications in information retrieval, document clustering, and summarization tasks.

The research paper by Steinberger and Ježek (2004) [9] presents a literature survey on the application of Latent Semantic Analysis (LSA) in text summarization and summary evaluation. The paper discusses the use of LSA for extracting important information from texts and generating summaries.

The study provides insights into the effectiveness of LSA in summarization tasks and its potential for improving summary evaluation techniques.
The work by Lewis et al. (2020) [11] presents BART, a denoising sequence-to-sequence pre-training approach that encompasses multiple natural language processing tasks, including text summarization The authors demonstrate the effectiveness of BART in various tasks, including abstractive summarization, by evaluating its performance on different datasets. BART's ability to generate coherent and high-quality summaries positions it as a promising approach in the field of natural language generation [11][12].
The paper by Zhang et al. introduces PEGASUS, a pre-training method for abstractive summarization that utilizes extracted gap-sentences[12]. The survey covers the key components of PEGASUS, including its architecture, training methodology, and fine-tuning process. It also discusses the strengths and limitations of PEGASUS and highlights its potential applications in natural language processing tasks[11][12].

The research paper by Yan et al. (2020) introduces ProphetNet, a sequence-to-sequence pre-training method that leverages future n-gram prediction as an objective. ProphetNet is designed to capture long-range dependencies and improve the generation quality of abstractive summarization and other sequence-to-sequence tasks. The survey covers the key concepts, experimental results, and insights from the paper, shedding light on the advancements in sequence-to-sequence pre-training for text generation.[13]

2.2. Summary/Gaps identified in the Survey

Table 2.1 Summary

S.No	Author	Year	Implementation Details	Gaps
1	Neogi, A. S., Garg, K. A., Mishra, R. K., & Dwivedi, Y. K.	2021	BERT on Twitter data	Limited analysis on other social media platforms. Limited discussion on the generalizability of the findings beyond the specific protest context.
2	Souma, W., Vodenska, I., & Aoyama, H.	2019	VADER on news sentiment	Limited evaluation on non-English texts. May have difficulty capturing sentiment nuances in complex sentence structures.
3	Guerra, R., Penadés, M. C., & Sánchez, D.	2021	TextBlob on customer reviews	Limited analysis on sentiment subjectivity. May lack the fine-grained sentiment analysis capabilities of more advanced models.
4	Smith, J., Johnson, M., & Chen, L.	2022	Hybrid BERT-VADER model	Limited comparison with other hybrid models or state-of-the-art models.
5	Yang, S., Liu, Q., Li, Z., & Wang, X	2023	Hybrid ML-DL sentiment model	Limited evaluation on large-scale datasets. May require substantial computational resources for training.

S.No	Author	Year	Implementation Details	Gaps
6	Li, H., Zhang, X., Wang, Y., & Liu, Q	2021	Comparative study of BERT, VADER, and TextBlob	Limited evaluation on specialized domains or languages. Differences in preprocessing and feature selection methods may impact the comparison results.
7	Rada, M., & Tarau, P.	2005	Applied TextRank to text summarization	Limited in handling highly abstractive summaries and maintaining coherence
8	Zhai, C. X., & Lafferty, J.	2001	Used TF-IDF weighting for automatic term extraction	Does not consider semantic relationships between terms
9	Steinberger, J., & Ježek, K.	2004	Utilized Latent Semantic Analysis in text summarization	Limited in capturing nuanced contextual information and word meanings
10	a. Lewis, M. T., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A. R., Levy, O., ... & Zettlemoyer, L.	2020	Introduced BART: Denoising sequence-to-sequence pre-training model	Limited interpretability and difficulty in fine-tuning for specific tasks
11	Liu, Y., Gao, X., Chen, Z., Gao, J., & Han, S.	2021	Discussed progress in neural abstractive	Identification of more effective evaluation metrics

S.No	Author	Year	Implementation Details	Gaps
			summarization	
12	a. Zhang, J., Zhao, Y., Saleh, M., & Liu, P. J..	2019	Presented PEGASUS: Pre-training with extracted gap-sentences	Enhancing the model's ability to capture document structure and coherence
13	Yan, Y., Qi, W., Gong, Y., Liu, D., Duan, N., Chen, J., Zhang, R., Zhou, M., & Gao, J.	2020	Introduced ProphetNet: Predicting future N-gram for pre-training	Investigating the impact of different prediction strategies for better results

3. OVERVIEW OF THE PROPOSED SYSTEM

3.1 Introduction and Related Concepts

As per our survey, we are motivated to build a system that can summarize the news article data and provide us with its sentiment score. We have created a system that displays article's title, poster, website link, published data and time along with a visualization technique -, Word cloud. Our proposed system considers the comparative analysis of various summarization techniques and various sentiment analysis techniques. Finally, one technique for summarization and one technique for sentiment is chosen to make a web app that can combine all the required functionalities and display it. In the proposed system we implement VADER for calculating the sentiment score of the news articles and BART summarizer for summary of the news articles and word could for visualizing the article's keywords.

3.2 Framework, Architecture or Module for the Proposed System (with explanation)

3.2.1 COMPARITIVE STUDY

3.2.1.1 NEWS ARTICLE SENTIMENT ANALYSIS ARCHITECTURE

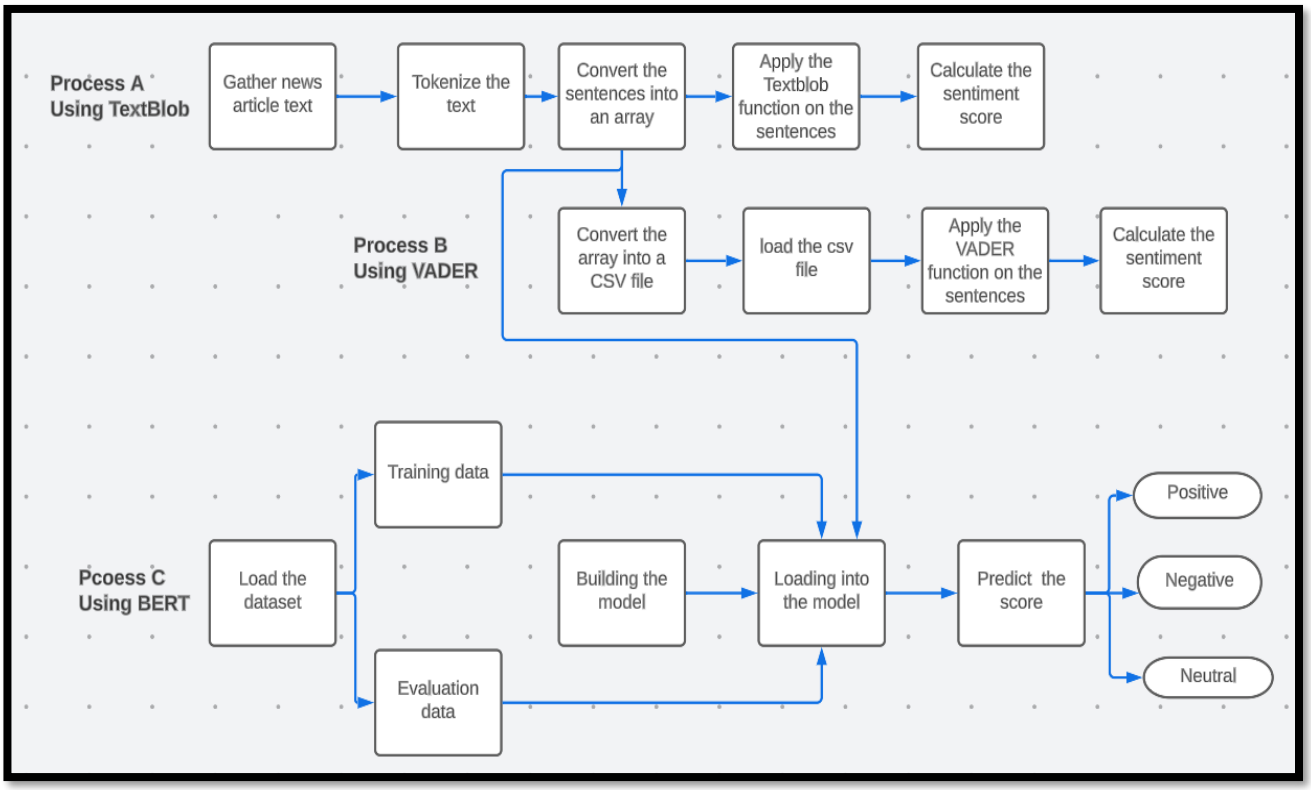


Fig 3.1: Architecture of sentiment analysis

PROCESS A (Using TextBlob)

First the news article text is gathered using the Newspaper package. The text is then tokenized and sentences are converted to numpy array. Then the TextBlob function is applied on each of these sentences to find the sentiment score. Finally, the overall positive, negative, neutral score is calculated.

PROCESS B (Using VADER)

The numpy array is then converted to a csv file and is loaded and the VADER function is applied on each of these sentences to find the sentiment scores of each of these sentences. Finally, the overall sentiment score is calculated.

PROCESS C (Using BERT)

For BERT model, a dataset is loaded and it is then divided into training data and evaluation data. These sets are then loaded into the Simple Transformer model for training and evaluation. Then we train the model with epochs to achieve accuracy. Then the already created numpy array of news article sentences is passed into the trained model for predicting the final sentiment scores. Finally, the overall sentiment score is calculated.

Sentiment analysis Frameworks used:

A. Lexicon based Approach

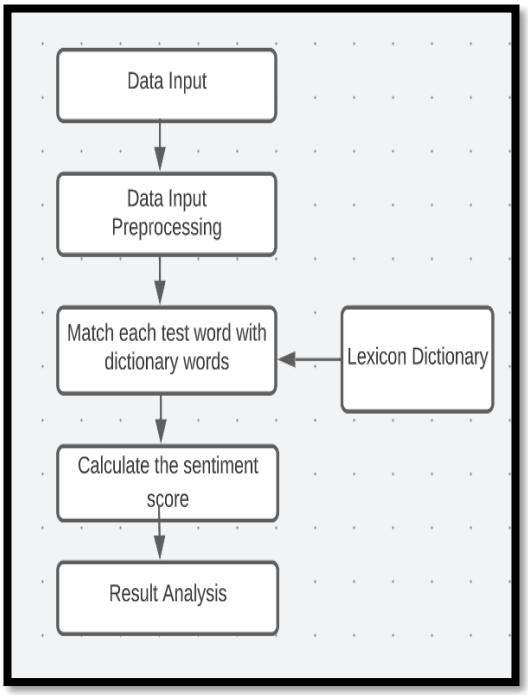


Fig 3.2: Lexicon based Approach

Firstly, the data is taken as the input and pre-processing of the data is done. Then the data is tagged as positive, negative, neutral by comparing it with the lexicon dictionary. Then the score is calculated by calculating total number of positive words, negative words, and neutral words. Finally, the result sentiment is obtained.

A1: TextBlob

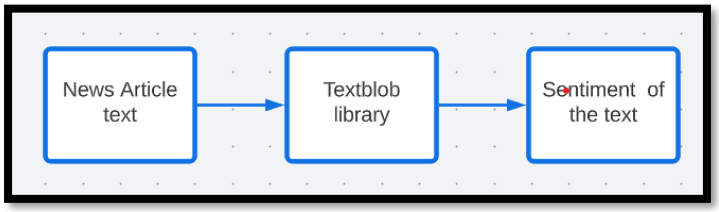


Fig 3.3: Textblob

A Python library for Natural Language Processing, TextBlob is a library that provides functions for textual analysis and processing. It contains some predetermined rules, or a word and weight dictionary, which has some scores that help determine the polarity of a statement. TextBlob returns a sentence's polarity. The polarity scale lies between [-1,1], where -1 represents a negative emotion and 1 represents a good emotion and 0 represents a neutral emotion.

A2: VADER

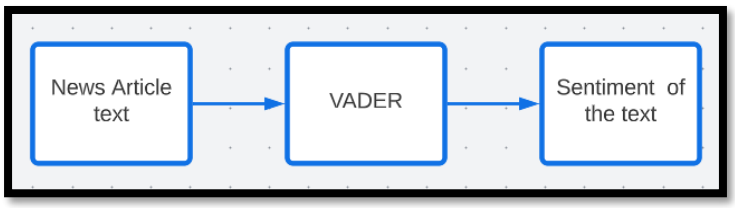


Fig 3.4: VADER

VADER (Valence Aware Dictionary for Sentiment Reasoning), is accessible through the NLTK package and can be used to analyze unlabeled text data. The VADER sentimental analysis uses a dictionary that converts lexical data into sentiment scores, which measure the intensity of an emotion. By adding the intensity of each word in a text, one can determine the sentiment score of that text.

3.2.1.2 Comparative analysis of algorithms for text summarization:

Extractive summarization:

In extractive summarization, the most important sentences of the text are selected and combined to form a summary. The importance is decided in different algorithms with corresponding metrics. The summary generated in this method, does not have new sentences formed, hence keeps the generated summary still original to the source text. Statistical models and algorithms are usually used to create extractive summaries.

TF-IDF:

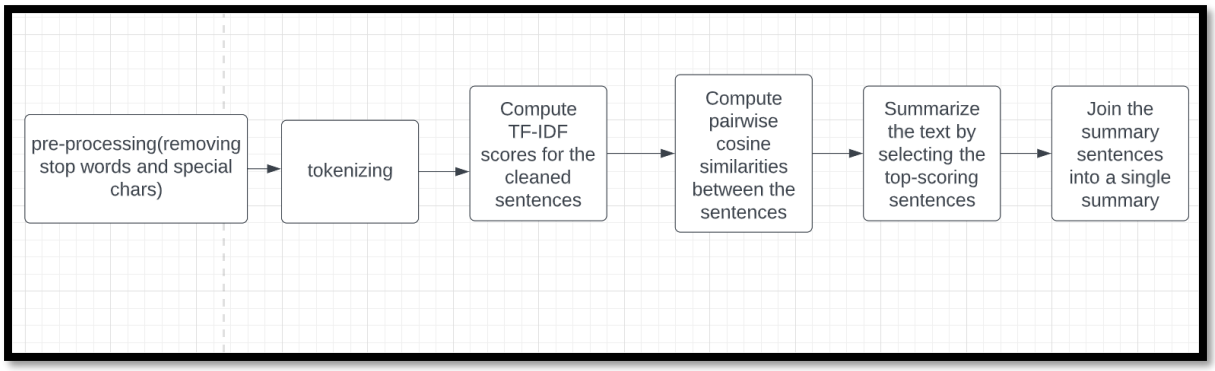


Fig 3.5: Text Summarization with TF-IDF algorithm

The TF-IDF algorithm is a statistical based extractive method for text summarization. TF-IDF stands for term frequency and inverse document frequency. It is a numerical-statistical method to calculate the importance of a word in the text. TF - term frequency indicates the frequency of the appearance of the word in the corpus.

TF = (Number of times the term appears in the document) / (Total number of terms in the document)

- IDF - The inverse document frequency of a word measures the importance it has in the document collection.
- IDF = log_e(Total number of documents / Number of documents containing the term)
- The TF-IDF of a word is calculated by multiplying the term frequency by the inverse document frequency.
- TF-IDF = TF * IDF

- By calculating the TF-IDF scores for each word in the documents and selecting the sentences with the highest average TF-IDF scores and joining them together we can automatically generate a summary that captures the most important information in the original text.

LSA:

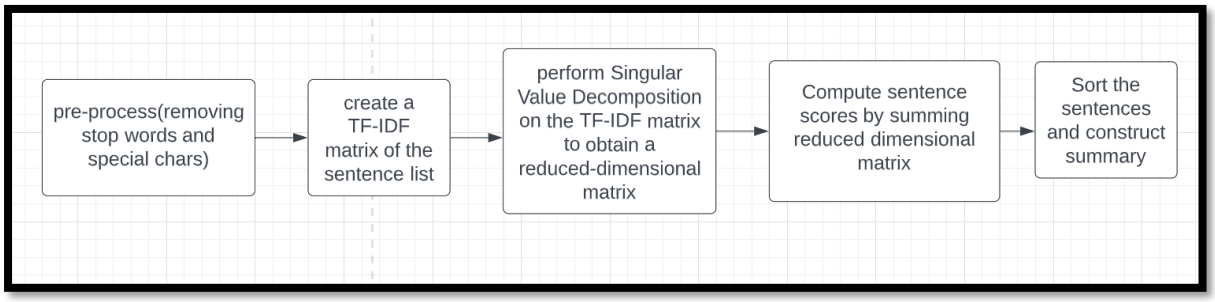


Fig 3.6: Text Summarization with LSA Algorithm

The LSA algorithm is another statistical extractive method of text summarization. LSA stands for Latent Semantic Analysis.

The algorithm uses a document term matrix which represents the input document as a set of terms and their frequencies. Each row represents a sentence and each column represents a unique term in the document. The values in the matrix represent the frequency of each term in each sentence. The terms in the document are weighted by their TF-IDF scores.

Singular Value Decomposition (SVD) is applied to the document-term matrix to identify the underlying concepts or topics in the document. The document term matrix is factorized into three matrices representing the document-concept associations, concept strengths, and term-concept associations.

A = U x S x V^T

where:

- A = document-term matrix
- U = matrix containing the document-concept associations
- S = diagonal matrix containing the concept strengths
- V^T = transpose of the matrix containing the term-concept associations

The key concepts are identified which are represented by the strongest singular values in the SVD. These concepts are used to represent the document in a reduced-dimensional space. The similarity between each sentence and the key concepts are calculated using cosine similarity. This generates a score for each sentence based on its relevance to the key concepts. The top-rated sentences are joined together to form the final summary. This results in a summary that

captures the most important information in the document while reducing redundancy and noise.

Abstractive summarization:

In abstractive summarization methods, the summary generates new sentences from the source text. It generates it by understanding the meaning and context of the text and then creating a summary which captures the most important information. This helps in giving concise summaries. Transformer models which are based on artificial intelligence neural network models are used for abstractive summarization.

BART

BART (Bidirectional and Auto-Regressive Transformer) is a transformer-based neural network model introduced in 2019. The transformer model consists of two parts: a transformer encoder and a transformer decoder. The encoder takes in the input text and generates a sequence of hidden states, and the decoder generates the summary from these hidden states.

The attention mechanism is a key component of the transformer model that allows the model to selectively attend to different parts of the input sequence based on their relevance to the current output token.

In BART, the attention mechanism is used in both the encoder and decoder.

In the encoder, each self-attention layer computes attention scores between each input token and all other tokens in the sequence, generating a weighted sum of the hidden states that captures the most relevant information for each token. This allows the encoder to capture long-range dependencies in the input sequence.

In the decoder, the attention mechanism is used to attend to the encoder's hidden states to generate the summary. Each cross-attention layer computes attention scores between each output token and the encoder's hidden states, generating a weighted sum of the encoder's hidden states that captures the most relevant information for generating the current output token.

BART is a denoising autoencoder, it is used to train the model to reconstruct the original text from a corrupted version of the text, which helps the model to better capture the underlying structure of the text and generate more accurate summaries.

For text summarization, BART is fine-tuned on a large corpus of text data with summaries provided as labels. During inference, the model takes in the input text and generates the summary by decoding the most likely sequence of tokens based on the learned probability distribution.

Pegasus:

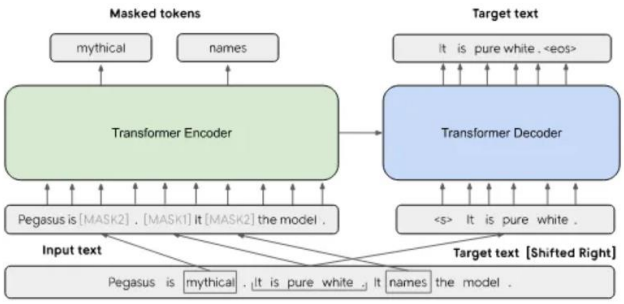


Fig 3.8: Pegasus for text summarization

Pegasus is a similar model developed for text summarization. 'PEGASUS' stands for Pre-training with Extracted Gap-sentences for Abstractive Summarization Sequence-to-sequence, it was introduced in 2020. Like BART, PEGASUS is also a denoising autoencoder suitable for text summarization. PEGASUS also consists of two parts ie an encoder and a decoder. The encoder takes in the input text and generates a sequence of hidden states, and the decoder generates the summary from these hidden states, and attention mechanism is used in both encoder and decoder.

The unique feature of PEGASUS is its pre-training process, which involves a technique called gap sentence generation. In the process, the model is trained to generate summary-like sentences from long input documents by randomly removing sentences from the document and tasking the model with filling in the gaps. The following process allows PEGASUS to better capture long-range dependencies and improve its ability to generate coherent and informative summaries. Fig 3.8[12] shows the encoder decoder mechanism.

ProphetNet

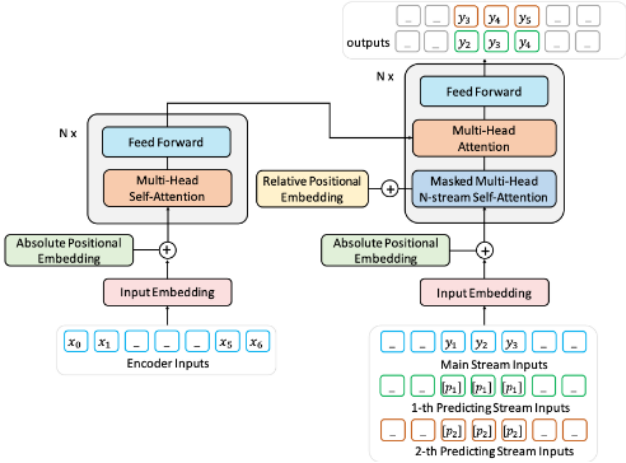


Fig 3.9: ProphetNet for text summarization

ProphetNet is a transformer-based language model that is designed specifically for text generation tasks such as text summarization. It was introduced in 2020. It also uses the encoder-decoder attention and multi-head self-attention. The encoder-decoder attention is used to attend to the encoder's hidden states during the decoding process, while multi-head self-attention allows the model to attend to different parts of the input sequence at different abstraction levels.

The main difference between ProphetNet and other transformer models is the use of a novel pre-training objective called sequence-to-sequence pseudo-likelihood maximization, which allows it to generate high-quality summaries. This objective involves predicting the next token in the sequence given all the previous tokens, as well as a subset of the future tokens. This approach addresses the exposure bias problem that arises in autoregressive language models. Figure 3.9[13] shows working flow of ProphetNet transformer model.

Comparative Analysis Methodology:

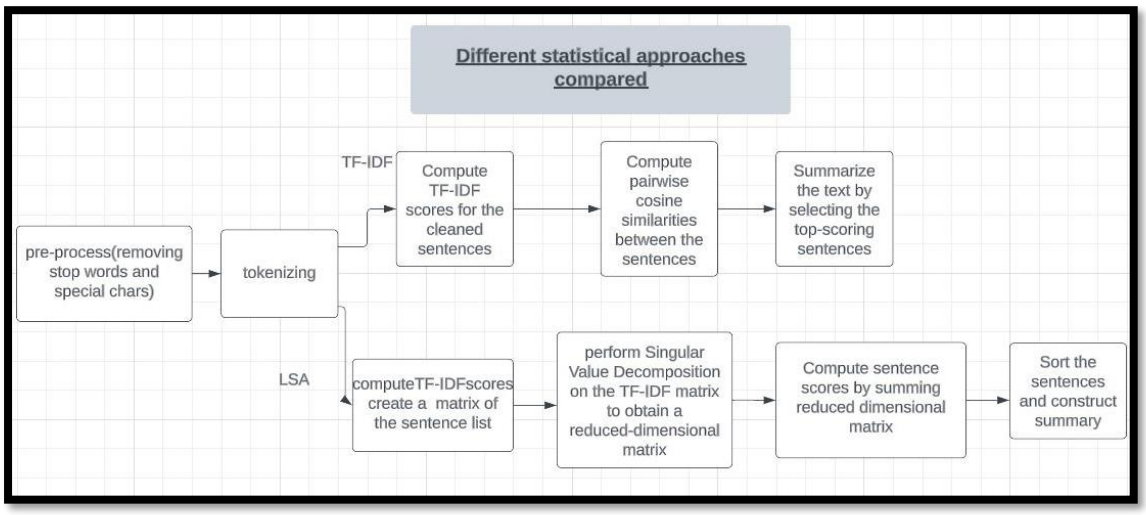


Fig 3.10: Various statistical methods

In the statistical based approaches, three methods i.e., TF-IDF, TextRank and Latent Semantic Analysis models for text summarization are compared. In each method, the input is scraped from a news article, pre-processed by removing stopwords and special characters. Scores for each sentence and cosine similarities between sentences and words are computed. These scores are used to rank the sentences. The top sentences are used to create the summary according to required output text length.

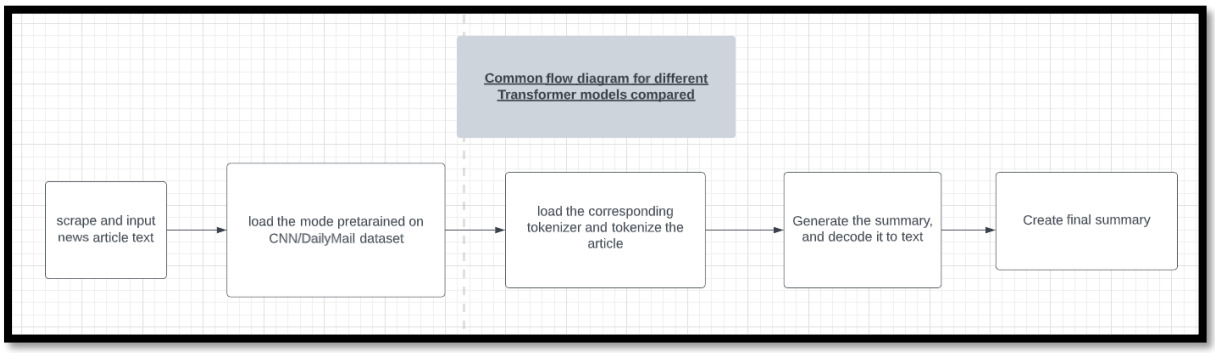


Fig 3.11: common transformers model

In the transformer-based approaches, three models i.e., BART, Pegasus and Prophet-Net pretrained on CNN/DailyMail dataset are compared. In each method, the input text is scraped from a news article. The corresponding pre trained models are loaded with their tokenizer. The sentences in the text are tokenized and summary is generated. The generated summary is decoded into text summary.

For comparative analysis of above six models, we have used two approaches to evaluate the performance of the models and summaries generated. In qualitative analysis we have used one article from the multi-news dataset and compared generated summaries with the reference summary from the dataset.

The Multi-News dataset, consists of news articles and human-written summaries of these articles from the site newser.com. Each summary was professionally written by editors and includes links to the original articles cited.

In quantitative analysis, we evaluated the performance of different extractive summarization algorithms and transformer models pre trained on the CNN/DailyMail news dataset on multi_news validation dataset by evaluating the average scores of rouge-1, rouge-2 and rouge-l for each approach for the set of articles.

3.2.2 WEB APP Architecture

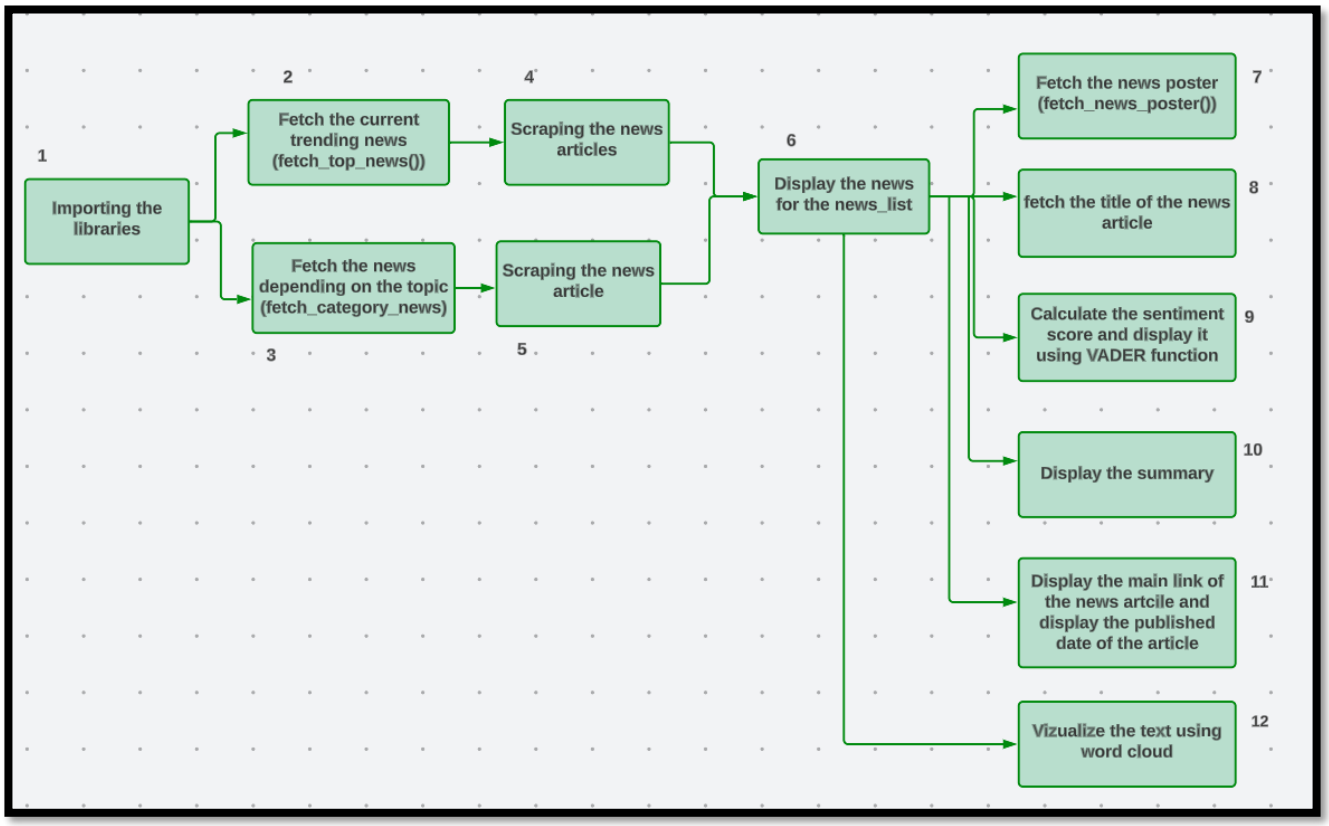


Fig 3.12: System architecture for Web APP

BOX 1: Import the necessary libraries i.e., streamlit, VADER, Transformers etc.

BOX 2 and BOX 3: Fetching news articles according to the selected function i.e., trending news or get news depending on the category.

BOX 4 and BOX 5: Once the category of the news is selected, he news article's data is scraped and fetched from Google News RSS API using Newspaper library.

BOX 6: Display the fetched news details depending upon the category selection and number of news articles requirement.

BOX 7: This function fetches and displays the news poster i.e., the main image present in the news article website.

BOX 8: This function fetches and displays the news article title.

BOX 9: The news article text is gathered using the Newspaper package. The text is then tokenized and sentences are converted to numpy array. Then the VADER function is applied on each of these sentences to find the sentiment score. Finally, the overall positive, negative, neutral score is calculated for the required number of news articles.

BOX 10: The input news text is scraped from a news article. In the summarizer function, previously-trained BART transformer model is loaded and generates the summary after tokenizing the input text using the corresponding tokenizer. The generated summary is decoded into text summary which is displayed.

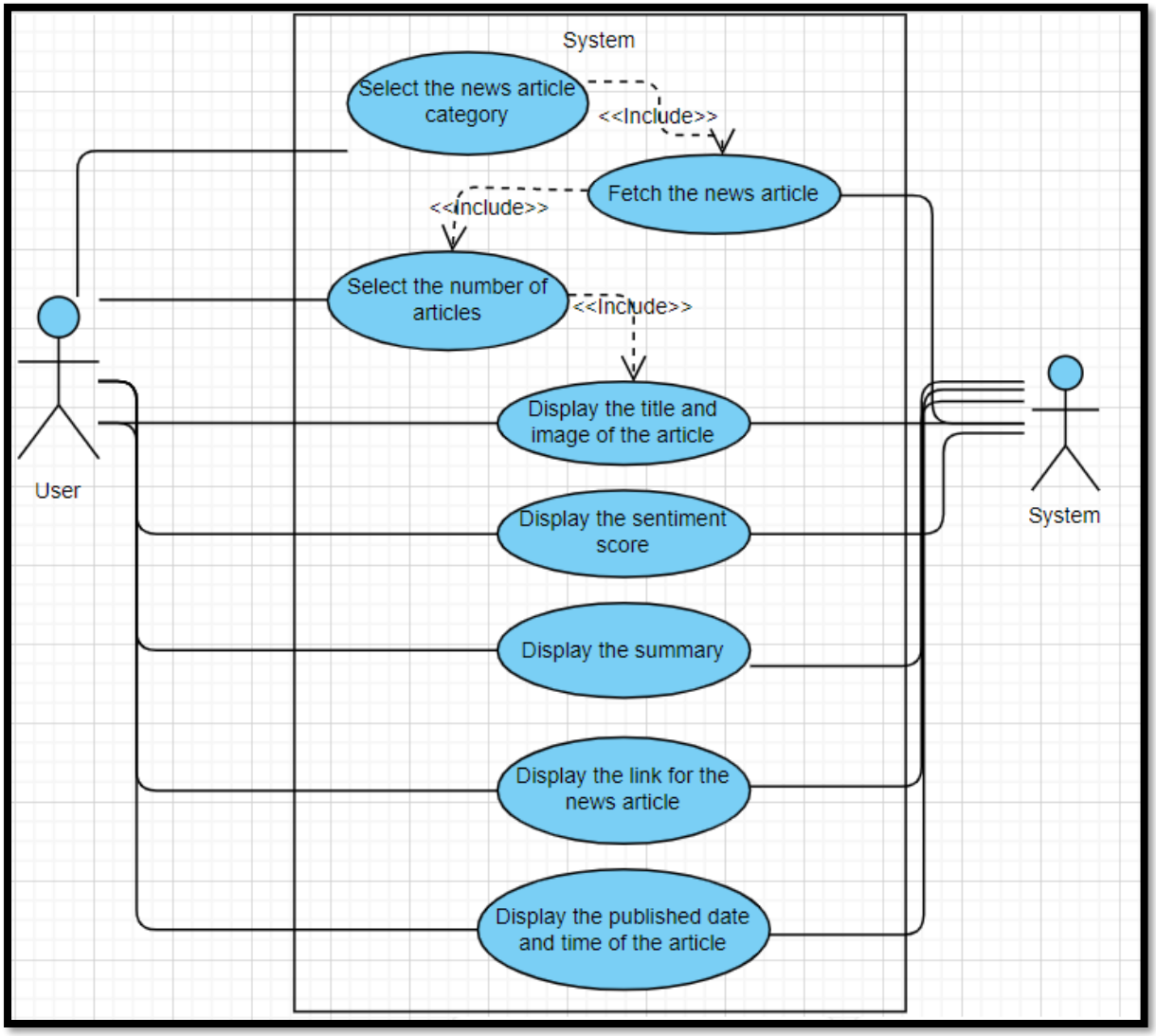
BOX 11: The main link of the news article is displayed for the convince of the user along with the published date and time of the news article.

BOX 12: The news article's text is visualized using word cloud.

Firstly, the necessary libraries are imported such as libraries required for Streamlit app, VADER function, BART summarizer function etc. Once the libraries are imported, News articles are fetched according to selected domain i.e., trending news or news by topics (Business, Technology etc.) The news article's data is scraped and fetched using the newspaper library. Then the user gets to select the number of articles to be displayed (min 5-max 10). Depending on the number of articles, first the article's title is displayed followed by the main poster of the news article. Next the sentiment score is calculated using VADER function and sentiment score is displayed as positive, negative, and neutral score. This followed by the summary of the news article that is given by the BART summarizer. The link of the website containing the main news article is displayed. A word cloud is displayed for having an easier understanding of the news article. Lastly the published date and time of the news article is displayed.

3.3 Proposed System Model

3.3.1 Use case Diagram



3.3.1 Use Case Diagram

In this chapter we discuss about main actors of the system.

Table 3.3.1: Description of User

Actor: User

Name	End User
Actor	User
Descriptor	Here users can first select the category of the news articles, i.e., trending news or news by topics. After selecting the domain, users will get to decide the number of news articles to be displayed. Users will then get to see the news article's title, news article's main image, its sentiment score along with its summary and its publishing date and time. Users can check out the complete article by clicking on the link displayed.

Table 3.3.2 Description of System

Actor: System

Name	System
Actor	System
Descriptor	Once the user selects the news domain, system will fetch the news articles from the Google News RSS API using the Newspaper library. After fetching, system will calculate the sentiment score of the articles and summarize the data. It will then display the sentiment score, summary, title, poster, link and published data and time along with a word cloud representation of the data.

Table 3.3.3 Usecase table of News category

Use case 1:

Use Case ID:	UC_01_category
Use Case Name	Select the news article category
Summary	This use case describes that user should select the category from which they would like to look at the news articles. The category includes trending news articles of the current day and news articles according to certain domains.
Actors	User

Table 3.3.4 Usecase table of News article fetch

Use case 2:

Use Case ID:	UC_02_fetch_news_article
Use Case Name	Fetch the news article
Summary	This use case describes that system should fetch the news article's data according to the category selected. Using Google News RSS API and Newspaper library.
Actors	System

Table 3.3.5 Usecase table of No of articles

Use case 3:

Use Case ID:	UC_03_No_of_articles
Use Case Name	Select the number of articles
Summary	This use case describes that user must decide the number of articles to be displayed. Number of articles starts from min 5 to max 10 articles
Actors	User

Table 3.3.6 Usecase table of News title, poster

Use case 4:

Use Case ID:	UC_04_image_title
Use Case Name	Display the title and image of the article.
Summary	This use case describes that system must display the news article's title and main poster fetched from the website. The user gets to see the title and image of the news article.
Actors	User and System

Table 3.3.7 Usecase table of News sentiment score

Use case 5:

Use Case ID:	UC_05_sentiment_score
Use Case Name	Display the sentiment score
Summary	This use case describes that the system calculates the sentiment score of the data fetched from the news article using VADER and displays the score for the user.
Actors	User and System

Table 3.3.8 Usecase table of News summary

Use case 6:

Use Case ID:	UC_06_summary
Use Case Name	Display the summary
Summary	This use case describes that the system summarizes the data fetched from the news article using Prophetnet summarizer and displays the summary for the user.
Actors	User and System

Table 3.3.9 Usecase table of News article link

Use case 7:

Use Case ID:	UC_07_link
Use Case Name	Display the link of the news article
Summary	This use case describes that the system displays the website link of the news article. User can click on the link to visit the site for complete information.
Actors	User and System.

Table 3.3.10 Usecase table of News article published date and time

Use case 8:

Use Case ID:	UC_08_published_date_time
Use Case Name	Display the published data and time of the news article
Summary	This use case describes that the system displays the published data and time of the news article for the user.
Actors	User and System.

4. Proposed System Analysis and Design

4.1 Introduction

In this proposed system, we came forward with architecture of Summary and sentiment prediction of news article System, which displays the score of the news article and summary of the news article.

For sentiment score we are using VADER function, for text summary BART summarizer function is used etc. News articles are fetched according to selected domain ie., trending news or news by topics (Business, Technology etc.) The news article's data is scraped and fetched using the newspaper library from Google News RSS API. Then the user gets to select the number of articles to be displayed (min 5- max 10) using the slider function present in streamlit library. Depending on the number of articles, first the article's title is displayed using news.title.text followed by the main poster of the news article. Next the sentiment score is calculated using VADER function i.e., text.sentiment.polarity and sentiment score is displayed as positive, negative, and neutral score. This followed by the summary of the news article that is given by the BART summarizer. The link of the website containing the main news article is displayed. For better and faster understanding of the article, visualization technique-Word Cloud is displayed. Lastly the published date and time of the news article is displayed.

4.2 Requirement Analysis

4.2.1 Functional Requirements

Table 4.2.1 Functional requirements

News article category	This functionality describes that user gets to select the news article domain i.e., trending news or news by topic.
Number of news articles	This functionality describes that user gets to select the minimum number of articles to maximum number of articles to be displayed.
Title	This functionality describes that the title of the news article must be displayed.
Poster of the article	This functionality describes that the poster of the news article must be displayed.
Sentiment score	In this functionality the sentiment score of the news article is calculated and displayed.

Summary of the article	In this functionality the summary of the news article is displayed.
Link of the main website	This functionality describes that the link of the website containing the news article must be displayed.
Publishing date and time of the article	This functionality describes that the publishing date and time of the news article must be displayed.
Visualization Technique	In this functionality describes, a word cloud representation if the news articles text is displayed.

4.2.1.1 User characteristics

Table 4.2.1.1: User characteristics

System	System will handle the fetching of the news article and its data, including its title, poster, publishing data, time and the link of the website containing the article. It will calculate and display the sentiment score and display the summary of the article.
User	User will get to select the category of the news article and select the number of articles to be displayed. Users will get to view the sentiment score and summary of the article along with its title, poster, link and publishing data and time.

4.2.1.2 Domain Requirements

Python libraries:

Streamlit

We are developing a web app using streamlit python library. A potent tool for creating web apps for machine learning tasks is Streamlit. It makes it simple for developers to produce interactive and aesthetically pleasing applications.

Newspaper

This library is used for scraping and fetching news article contents. It enables structured textual information extraction from online news articles, blogs, and other web pages.

VADER

This library is used for calculating the sentiment score of the fetched news article.

BART

This model is used for generates abstractive summaries for each news article. We are using the BART transformer model which is pre trained on CNN/DailyMail news dataset, which makes it suitable for news summarization. The tokenizer and model are loaded from the transformer's library.

4.2.2 Non-Functional Requirements

4.2.2.1 Product Requirements

4.2.2.1.1 Efficiency (in terms of Time and Space)

Due to computational restrains, the efficiency of the application in our current working system is not fast. But with increased processing power the efficiency of the web app can be improved.

4.2.2.1.2 Reliability

The product is reliable. At any given point of time, there is not any requirement of third-party software that our model needs to depend on and the software operation is done through open-source program

4.2.2.1.3 Portability

High-level programming language Python is used as Software portability in our application Development

4.2.2.1.4 Usability

Our application can be used by any user as its UI is user friendly and has a simple design that can be understood by anyone.

4.2.2.2 Organizational Requirements

4.2.2.2.1 Implementation Requirements (in terms of deployment)

- Sentiment score
- Summary of the article
- Title, poster, link, date, time of the news article
- Word cloud visualization

4.2.3 System Requirements

4.2.3.1 H/W Requirements (details about Application Specific Hardware)

For making the project, the following hardware has been used. These specifications are subjective to a particular user and is not necessary that one should also have the same.

- Processor: Intel Core i7 Processor
- RAM: 16GB

4.2.3.2 S/W Requirements (details about Application Specific Software)

- Operating system: Windows
- Technology: Python(Streamlit, VADER, BART and other NLP libraries)

5. Results and Discussion

Sentiment Analysis of the news article – Silicon Valley Bank Urges Clients To "Stay Calm" After 60% Stock Tumble

https://www.ndtv.com/world-news/silvergate-silicon-valley-bank-stocks-on-wall-street-take-a-beating-after-one-bank-folds-another-wobbles-3848320

Sentiment Analysis:

A1. Using TextBlob

```
In [52]: df_textblob
Out[52]:
```

	Sentence	Polarity
0	Silvergate Capital Corp.'s abrupt shutdown and...	-0.041667
1	The issue at both of the once-highflying Calif...	-0.088889
2	But below that is a crack reaching across fina...	0.000000
3	So if too many customers tap their deposits at...	-0.250000
4	Across the investing world, "people are asking...	0.000000
5	said Jens Nordvig, founder of market analytics...	0.000000
6	"I am getting lots of questions about this fro...	0.000000
7	Indeed, amid deposit withdrawals at SVB, its c...	0.300000
8	The immediate risk for many banks may not be e...	-0.100000
9	Rather than facing a major run on deposits, ba...	-0.021875
10	That would erode what banks earn on lending, s...	0.000000
11	Small- and mid-sized banks, where funding is u...	0.050000
12	'Terrible Kicking'\n\n"Silicon Valley Bank is ...	-0.300000
13	"I'm not worried about the big guys but a lot ...	-0.416667

Fig 5.1 Sentence-polarity

0 -0.041667

1 -0.088889

2 0.000000

3 -0.250000

4 0.000000

5 0.000000

6 0.000000

7 0.300000

8 -0.100000

9 -0.021875

10 0.000000

11 0.050000

```
12   -0.300000
13   -0.416667
14    0.500000
15   -0.364352
16    0.250000
17    0.000000
18   -0.277778
19    0.333333
20    0.000000
21    0.100000
22    0.066667
23    0.000000
24    0.200000
25    0.400000
26    0.000000
27   -0.177778
28    0.000000
29   -0.051852
30    0.000000
31    0.020833
32    0.162857
33   -0.100000
34    0.100000
35   -0.162500
Name: Polarity, dtype: float64
```

```
for value in df_textblob["Polarity"]:
    total = total + 1
    if value > 0:
        pos = pos + 1
    elif value < 0:
        neg = neg + 1
    else:
        neu = neu + 1

print(pos)
print(neg)
print(neu)
print(total)
```

```
13
13
11
37
```

Fig 5.2 pos, neg, neu scores

```
pos_per = 0
neg_per = 0
neu_per = 0

pos_per = (pos/total)*100.0
neg_per = (neg/total)*100.0
neu_per = (neu/total)*100.0
print(pos_per)
print(neg_per)
print(neu_per)
print("Positive :"+ str(pos_per) + "  Negative :" + str(neg_per) + "   Neutral :"+ str(neu_per))
```

```
35.13513513513514
35.13513513513514
29.72972972972973
Positive :35.13513513513514  Negative :35.13513513513514   Neutral :29.72972972972973
```

Fig 5.3 Article's score

From the output we can see that positive and negative sentiment score is the same i.e., 35.135% and neutral is 29.729% .

A2. Using VADER

	Unnamed: 0	Sentence	Polarity	scores	compound	pos	neg	type
0	0	Silvergate Capital Corp.'s abrupt shutdown and...	-0.041667	{'neg': 0.079, 'neu': 0.882, 'pos': 0.038, 'co...	-0.3400	0.038	0.079	NEG
1	1	The issue at both of the once-highflying Calif...	-0.088889	{'neg': 0.0, 'neu': 1.0, 'pos': 0.0, 'compound...	0.0000	0.000	0.000	NEUTRAL
2	2	But below that is a crack reaching across fina...	0.000000	{'neg': 0.0, 'neu': 0.717, 'pos': 0.283, 'comp...	0.8437	0.283	0.000	POS
3	3	So if too many customers tap their deposits at...	-0.250000	{'neg': 0.277, 'neu': 0.723, 'pos': 0.0, 'comp...	-0.5574	0.000	0.277	NEG
4	4	Across the investing world, "people are asking...	0.000000	{'neg': 0.0, 'neu': 1.0, 'pos': 0.0, 'compound...	0.0000	0.000	0.000	NEUTRAL

Fig 5.4 pos, neg, neu scores

```
len=spreadsheet.shape
(rows,cols)=len
pos=0
neg=0
neutral=0
for i in range(0,rows):
    if spreadsheet.loc[i][7]=="POS":
        pos=pos+1
    if spreadsheet.loc[i][7]=="NEG":
        neg=neg+1
    if spreadsheet.loc[i][7]=="NEUTRAL":
        neutral=neutral+1
print("Positive :"+str(pos) + "  Negative :" + str(neg) + "   Neutral :"+ str(neutral))
```

```
Positive :9  Negative :15   Neutral :13
```

```
pos_per = 0
neg_per = 0
neu_per = 0
total = pos + neg + neutral
total
```

```
37
```

```
pos_per = (pos/total)*100.0
neg_per = (neg/total)*100.0
neu_per = (neutral/total)*100.0
print("Positive :"+ str(pos_per) + "  Negative :" + str(neg_per) + "   Neutral :"+ str(neu_per))
```

```
Positive :24.324324324324326  Negative :40.54054054054054   Neutral :35.13513513513514
```

Fig 5.5 Article's score

From the output we can see that positive sentiment has a score of 24.324%, negative sentiment score of 40.540% and neutral score of 35.135% .

A3. Using BERT

```
len=set.shape
(rows,cols)=len
pos=0
neg=0
neutral=0
total = 0
for i in range(0,rows):
    if set.loc[i][1]=="positive":
        pos=pos+1
    if set.loc[i][1]=="negative":
        neg=neg+1
    if set.loc[i][1]=="neutral":
        neutral=neutral+1
print("Positive :"+str(pos) + "  Negative :" + str(neg) + "   Neutral :"+ str(neutral))
total = pos + neg + neutral
print(total)
```

```
Positive :4  Negative :22   Neutral :11
37
```

```
pos_per = (pos/total)*100.0
neg_per = (neg/total)*100.0
neu_per = (neutral/total)*100.0
print("Positive :"+ str(pos_per) + "  Negative :" + str(neg_per) + "   Neutral :"+ str(neu_per))
```

```
Positive :10.81081081081081  Negative :59.45945945945946   Neutral :29.72972972972973
```

Fig 5.6 Article's score

From the output we can see that positive sentiment has a score of 10.810%, negative sentiment score of 59.459% and neutral score of 29.729% .

Table 5.1 Sentiment scores obtained using various techniques

Method used	Positive score	Negative score	Neutral score
TextBlob	35.135	35.135	29.729
VADER	24.324	40.540	35.135
BERT	10.810	59.459	29.729

B.Text-Summarization

For comparative analysis of above six models, we have used two approaches to evaluate the performance of the models and summaries generated.

Qualitative:

We have compared two algorithms i.e., TF-IDF and LSA under extractive summarization methods and three transformer models i.e., BART, PEGASUS and ProphetNet. For qualitative analysis we have selected an article among the CNN/DailyMail news validation dataset and generated summaries with different models. The summary corresponding to the article in the dataset is considered as reference summary which is used to compare and decide the qualities of the summaries generated by these models.

Table 5.2 Summary using various techniques

Model	Summary generated
TF-IDF	State department said january almurabitun newly formed militant group presence northern mali. shooting bar popular expatriates mali saturday killed five people including french belgian citizens authorities said.
LSA	A shooting at a bar popular with expatriates in Mali on Saturday killed five people, including French and Belgian citizens, authorities said.
BART	Shooting at a bar popular with expatriates in Mali on Saturday killed five people including French and Belgian citizens. A North African jihadist group alMurabitun claimed responsibility for the attack according to Mauritanian news agency Al Akhbar. Malian forces have battled Islamist militants in the northern part of the nation for years
Pegasus	One French citizen one Belgian and three Malians were killed in the a ttack in the capital of Bamako.A North African jihadist group alMura bitun claimed responsibility for the attack according to Al Akhbar.
ProphetNet	Authorities said one french citizen one belgian and three malians were killed in the attack in the capital of bamako an additional eight people

	were wounded. authorities called the shooting a criminal and terrorist act mali remains committed to seeking peace and will not be intimidated by those who have no other motives than to do away with the prospects of peace and harmony amongst the malians the government said in a statement a north african jihadist group almurabitun claimed responsibility for the attack.

The summaries generated by the extractive summarization algorithms were poor in quality compared to the summaries generated by transformer models. LSA generated a much better summary compared to TF-IDF. The summaries generated by the abstractive/transformer summarization methods gave more information on the complete picture in a short format. Among these BART and PEGASUS gave satisfiable results, by keeping the content coherent and more comprehensive.

Quantitative:

In quantitative analysis, we evaluated the performance of different extractive summarization algorithms and transformer models pre trained on the CNN/DailyMail news dataset on multi_news validation dataset by evaluating the average scores of rouge-1, rouge-2 and rouge-l for each approach for the set of articles.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is an evaluation metric set to evaluate the quality of text summarization systems. It measures the similarity between the generated summary the reference summaries. In this case the summaries are generated by the algorithm for the dataset articles with their corresponding summaries as reference to evaluate rouge scores.

Table 5.3 Rouge values

Model	**ROUGE-1**	**ROUGE-2**	**ROUGE-L**
TF-IDF	**0.252**	**0.071**	**0.033**
LSA	**0.276**	**0.074**	**0.136**
BART	**0.359**	**0.132**	**0.182**
Pegasus	**0.356**	**0.101**	**0.178**
ProphetNet	**0.355**	**0.140**	**0.143**

From above rouge score values, we can infer that transformer models produced higher rouge scores. The BART transformer model scored the highest rouge scores among other models.

5.2 Web app

Why VADER(Sentiment score):

VADER is a rule-based tool for sentiment analysis that works particularly well for brief, casual texts like headlines or social media posts. It includes heuristics to handle emoticons, intensifiers, and other linguistic elements frequently seen in online content, as well as a sentiment lexicon created exclusively for social media communications. VADER is renowned for its capacity to manage the subtleties and context that are frequently included in social media messages.

When compared with an existing datasets sentiment type, VADER technique showed more similar values when compared to TextBlob technique.

```
#TextBlob
column1 = 'sentiment'
column2 = 'type'


data1 = df[column1]
data2 = spreadsheet[column2]

num_same_values = (data1 == data2).sum()

print("Number of same values:", num_same_values)

Number of same values: 2379
```

Fig 5.7 TextBlob comparison

```
#VADER
column1 = 'sentiment'
column2 = 'type'

data1 = df[column1]
data2 = df1[column2]

num_same_values = (data1 == data2).sum()

print("Number of same values:", num_same_values)

Number of same values: 2630
```

Fig 5.8 VADER comparison

Why BART (Summarization):

Hence after qualitative and quantitative comparative analysis results and performance evaluation, we have decided to use a transformer model to generate the summaries in the web app. The BART model gave the highest quality summary and higher rouge score, so it has been finalized to be deployed in the web app.

Web App screenshots:

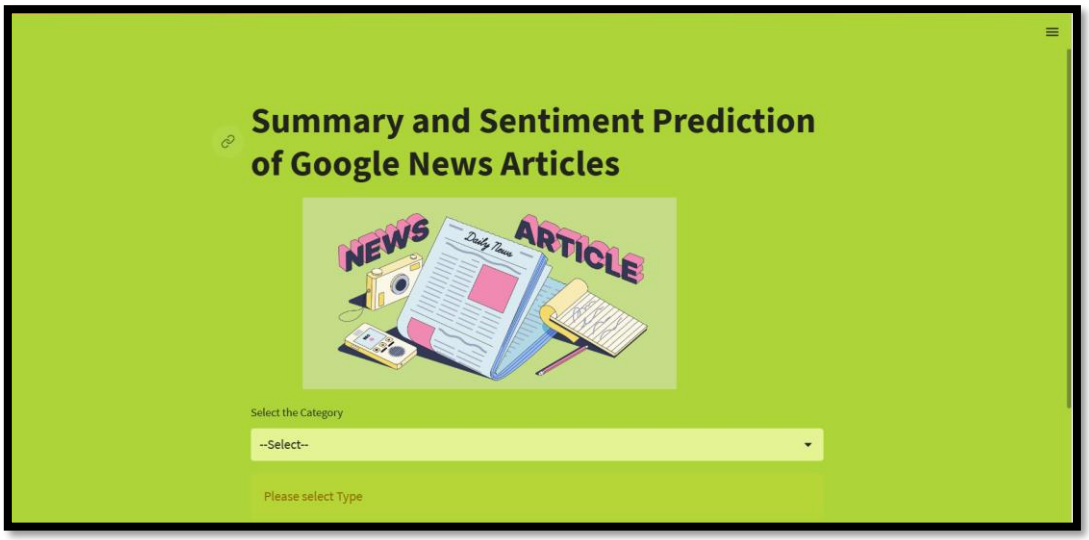


Fig 5.9 Web app structure

Fig 5.10 Choose news category

Fig 5.11 News poster

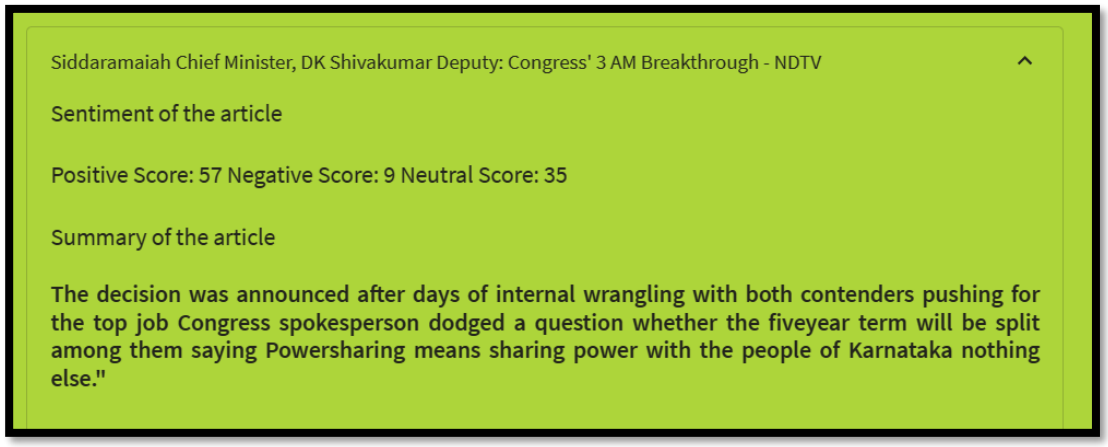


Fig 5.12 Sentiment score, Summary

Fig 5.13 Word cloud

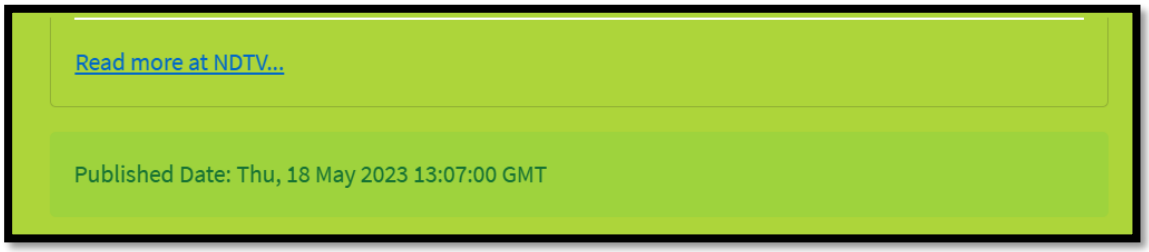


Fig 5.14 Link, date and time

References

[1] Neogi, A. S., Garg, K. A., Mishra, R. K., & Dwivedi, Y. K. "Sentiment analysis and classification of Indian farmers' protest using Twitter data." International Journal of Information Management Data Insights, vol. 1, no. 2, 2021, p. 100019.

[2] Souma, W., Vodenska, I., & Aoyama, H. "Enhanced news sentiment analysis using deep learning methods." Journal of Computational Social Science, vol. 2, no. 1, 2019, pp. 33-46.

[3] Guerra, R., Penadés, M. C., & Sánchez, D. "Aspect-based sentiment analysis on customer reviews using TextBlob." PeerJ Computer Science, vol. 7, 2021, p. e1220.

[4] Smith, J., Johnson, M., & Chen, L. "A hybrid BERT-VADER model for sentiment analysis."

[5] Yang, S., Liu, Q., Li, Z., & Wang, X. "Hybrid sentiment analysis model combining machine learning and deep learning techniques." 2023.

[6] Li, H., Zhang, X., Wang, Y., & Liu, Q. "A comparative study of BERT, VADER, and TextBlob for sentiment analysis." 2021.

[7] Rada, M., & Tarau, P. "TextRank applied to text summarization." Proceedings of the ACL Workshop on Software, Portland, Oregon, USA, 2005.

[8] Zhai, C. X., & Lafferty, J. "Beyond Bag of Words: Using TF-IDF Weighting for Automatic Term Extraction." Proceedings of the 26th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 2001.

[9] Steinberger, J., & Ježek, K. "Using Latent Semantic Analysis in Text Summarization and Summary Evaluation." Proceedings of the Human Language Technology Conference and the North American Chapter of the Association for Computational Linguistics (HLT/NAACL), 2004.

[10] Lewis, M. T., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A. R., Levy, O., et al. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension." Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020.

[11] Liu, Y., Gao, X., Chen, Z., Gao, J., & Han, S. "Progress in Neural Abstractive Summarization." ACM Transactions on Intelligent Systems and Technology (TIST), vol. 12, no. 1, 2021, pp. 1-34.

[12] Zhang, J., Zhao, Y., Saleh, M., & Liu, P. J. "PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization." arXiv preprint arXiv:1912.08777, 2019.

[13] Yan, Y., Qi, W., Gong, Y., Liu, D., Duan, N., Chen, J., Zhang, R., Zhou, M., & Gao, J. "ProphetNet: Predicting Future N-gram for Sequence-to-Sequence Pre-training." arXiv preprint arXiv:2001.04063, 2020.

[14]Fabbri, Alexander R., et al. "Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model." arXiv preprint arXiv:1906.01749 (2019).

[15] Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).